

CAPÍTULO 1

Algoritmos de Ordenamiento

El problema del ordenamiento es fundamental en ciencias de la computación. Un algoritmo de ordenamiento reorganiza los elementos de un arreglo o lista de acuerdo a una relación de orden. En este capítulo estudiaremos varios algoritmos clásicos, analizando su funcionamiento, correctitud y complejidad.

1.1 El problema del ordenamiento

Recordemos la definición formal del problema:

Problema del Ordenamiento

Entrada: Una secuencia de n números $\langle a_1, a_2, \dots, a_n \rangle$.

Salida: Una permutación (reordenamiento) $\langle a'_1, a'_2, \dots, a'_n \rangle$ de la secuencia de entrada tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Los números que deseamos ordenar también se conocen como **claves**. Aunque conceptualmente estamos ordenando una secuencia, la entrada viene en forma de un arreglo con n elementos.

1.1.1 Clasificación de algoritmos de ordenamiento

Los algoritmos de ordenamiento se pueden clasificar de varias formas:

- **Por comparación vs. sin comparación:** Los algoritmos basados en comparaciones (como Merge Sort, Quick Sort) solo usan comparaciones entre elementos. Los algoritmos sin comparación (como Counting Sort, Radix Sort) explotan propiedades de los datos.
- **Estables vs. inestables:** Un algoritmo es **estable** si mantiene el orden relativo de elementos con claves iguales.
- **In-place vs. con memoria auxiliar:** Un algoritmo es **in-place** si usa solo $O(1)$ memoria adicional.
- **Internos vs. externos:** El ordenamiento **interno** asume que todos los datos caben en memoria principal. El **externo** maneja datos que residen en almacenamiento secundario.

1.2 Insertion Sort

Insertion Sort es un algoritmo simple y eficiente para ordenar un número pequeño de elementos. Funciona de la manera en que muchas personas ordenan una mano de cartas:

1. Comenzamos con la mano izquierda vacía y las cartas boca abajo en la mesa.
2. Tomamos una carta a la vez de la mesa y la insertamos en la posición correcta en la mano izquierda.
3. Para encontrar la posición correcta, comparamos la carta con cada una de las cartas ya en la mano, de derecha a izquierda.

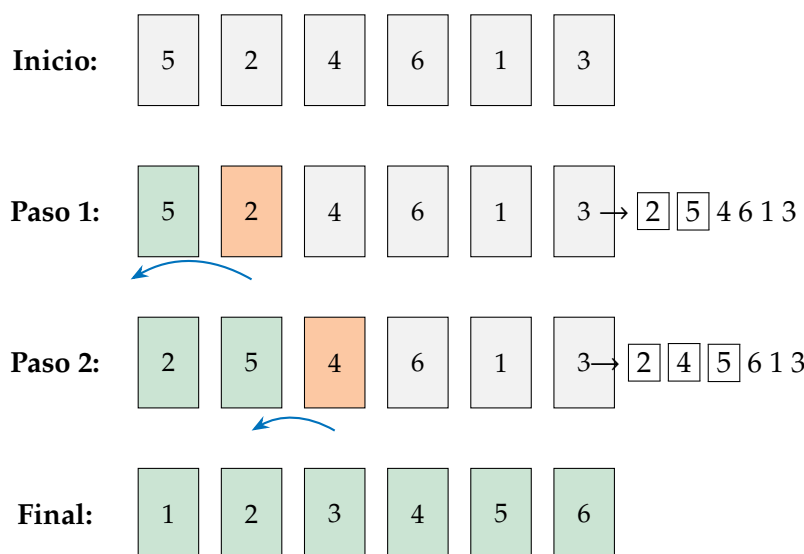


Figura 1.1: Funcionamiento de Insertion Sort. Los elementos verdes están ordenados, el naranja es el elemento actual siendo insertado.

1.2.1 Pseudocódigo

Algorithm 1 Insertion-Sort(A, n)

```

1: for  $j = 2$  to  $n$  do
2:    $key = A[j]$ 
3:   {Insertar  $A[j]$  en la secuencia ordenada  $A[1..j - 1]$ }
4:    $i = j - 1$ 
5:   while  $i > 0$  and  $A[i] > key$  do
6:      $A[i + 1] = A[i]$ 
7:      $i = i - 1$ 
8:   end while
9:    $A[i + 1] = key$ 
10: end for

```

1.2.2 Invariante de ciclo

Para demostrar que Insertion Sort es correcto, usamos el siguiente invariante de ciclo:

Invariante: Al inicio de cada iteración del ciclo **for**, el subarreglo $A[1..j - 1]$ consiste en los elementos originalmente en $A[1..j - 1]$, pero en orden.

- **Inicialización:** Antes de la primera iteración ($j = 2$), el subarreglo $A[1..1]$ consiste en un solo elemento, que trivialmente está ordenado.
- **Mantenimiento:** El cuerpo del ciclo mueve $A[j - 1]$, $A[j - 2]$, etc. una posición a la derecha hasta encontrar la posición correcta para $A[j]$ (líneas 5-8), momento en el cual inserta el valor de $A[j]$ (línea 9). El subarreglo $A[1..j]$ queda ordenado.

- **Terminación:** El ciclo termina cuando $j = n + 1$. Sustituyendo $n + 1$ por j en el invariante, tenemos que $A[1..n]$ está ordenado.

1.2.3 Análisis de complejidad

Teorema 1.1. *Insertion Sort tiene complejidad $O(n^2)$ en el peor caso y $O(n)$ en el mejor caso.*

Demostración. El ciclo **for** externo se ejecuta $n - 1$ veces. En el **peor caso** (arreglo en orden inverso), el ciclo **while** interno se ejecuta $j - 1$ veces para cada j :

$$\sum_{j=2}^n (j - 1) = 1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2} = \Theta(n^2)$$

En el **mejor caso** (arreglo ya ordenado), el ciclo **while** nunca se ejecuta, resultando en $\Theta(n)$. $\square$

1.3 Merge Sort

Merge Sort es un algoritmo que sigue el paradigma de **divide y vencerás**:

1. **Dividir:** Divide la secuencia de n elementos en dos subsecuencias de $n/2$ elementos cada una.
2. **Conquistar:** Ordena las dos subsecuencias recursivamente usando Merge Sort.
3. **Combinar:** Mezcla las dos subsecuencias ordenadas para producir la respuesta ordenada.

1.3.1 Pseudocódigo

Algorithm 2 Merge-Sort(A, p, r)

```

1: if  $p \geq r$  then
2:   return {Caso base: 0 o 1 elementos}
3: end if
4:  $q = \lfloor (p + r)/2 \rfloor$  {Punto medio}
5: Merge-Sort( $A, p, q$ ) {Ordenar mitad izquierda}
6: Merge-Sort( $A, q + 1, r$ ) {Ordenar mitad derecha}
7: Merge( $A, p, q, r$ ) {Combinar las mitades}
```

La operación clave es el procedimiento **MERGE**, que combina dos subarreglos ordenados $A[p..q]$ y $A[q + 1..r]$ en un solo subarreglo ordenado:

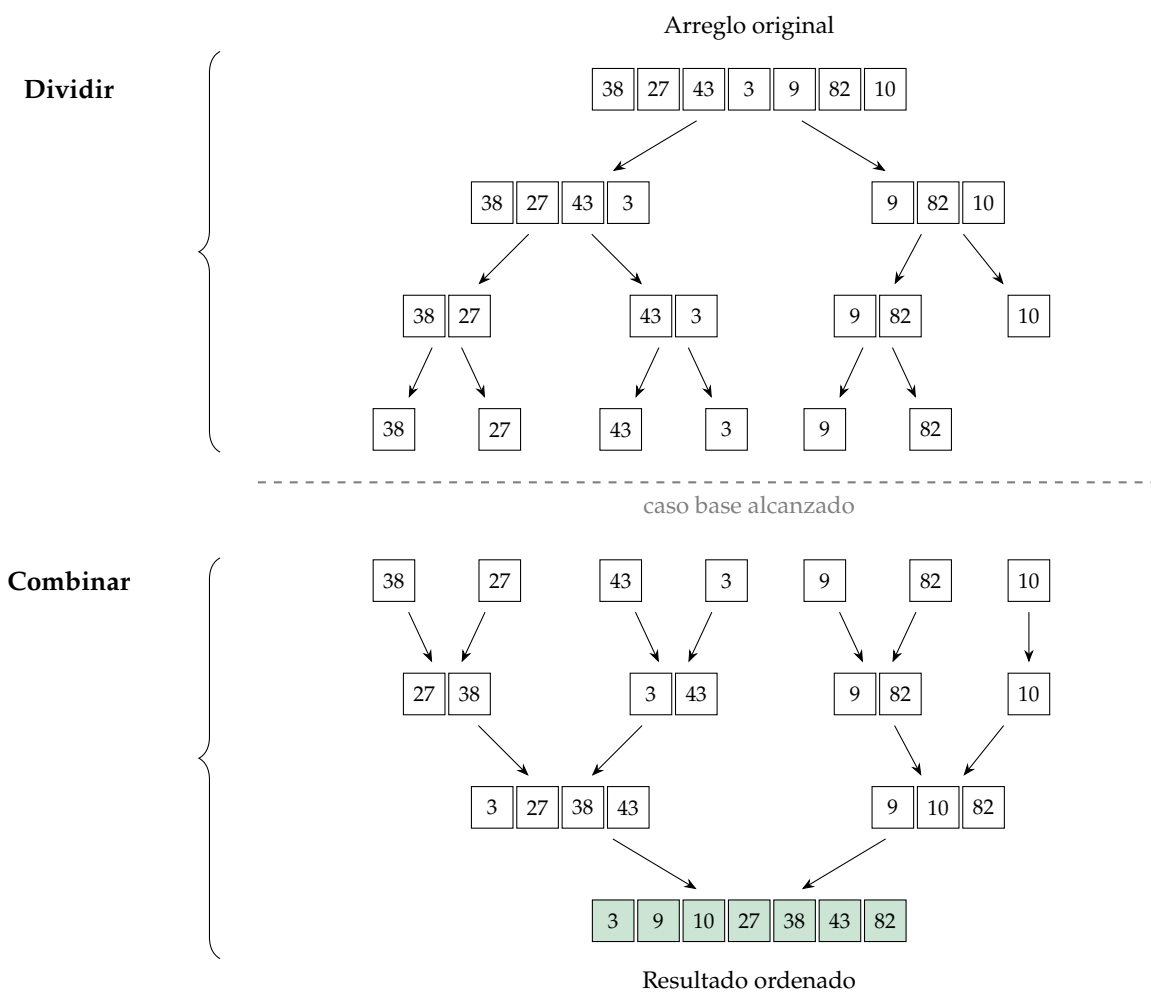


Figura 1.2: Merge Sort: fase de división (arriba) y fase de combinación (abajo). Los elementos se dividen recursivamente hasta llegar al caso base, luego se combinan en orden.

Algorithm 3 Merge(A, p, q, r)

```

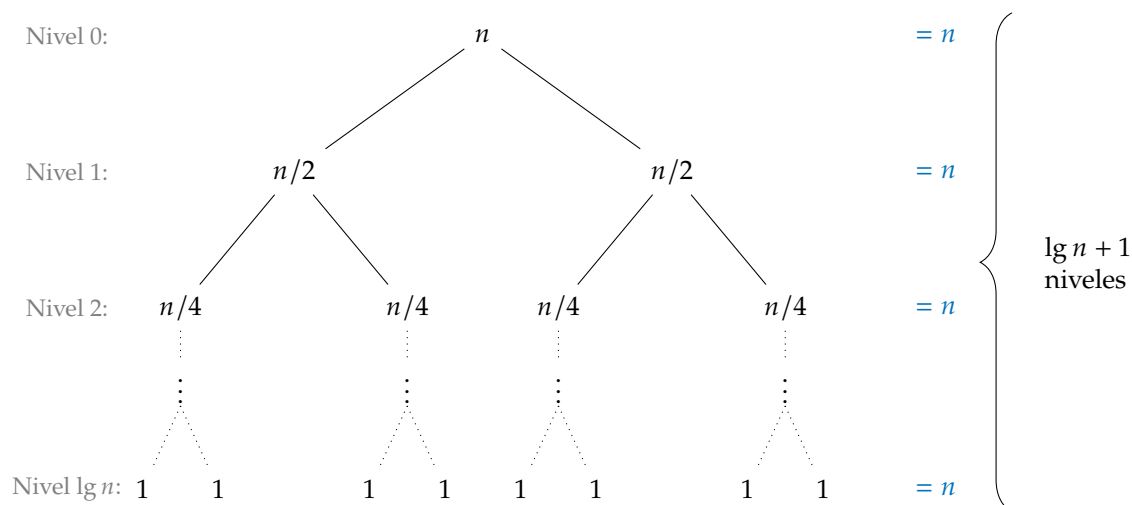
1:  $n_L = q - p + 1$  {Longitud de  $A[p..q]$ }
2:  $n_R = r - q$  {Longitud de  $A[q + 1..r]$ }
3: Crear arreglos  $L[1..n_L]$  y  $R[1..n_R]$ 
4: for  $i = 1$  to  $n_L$  do
5:    $L[i] = A[p + i - 1]$ 
6: end for
7: for  $j = 1$  to  $n_R$  do
8:    $R[j] = A[q + j]$ 
9: end for
10:  $i = 1, j = 1, k = p$ 
11: while  $i \leq n_L$  and  $j \leq n_R$  do
12:   if  $L[i] \leq R[j]$  then
13:      $A[k] = L[i]; i = i + 1$ 
14:   else
15:      $A[k] = R[j]; j = j + 1$ 
16:   end if
17:    $k = k + 1$ 
18: end while
19: while  $i \leq n_L$  do
20:    $A[k] = L[i]; i = i + 1; k = k + 1$ 
21: end while
22: while  $j \leq n_R$  do
23:    $A[k] = R[j]; j = j + 1; k = k + 1$ 
24: end while

```

1.3.2 Análisis de complejidad

El tiempo de ejecución de Merge Sort está dado por la recurrencia:

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ 2T(n/2) + \Theta(n) & \text{si } n > 1 \end{cases}$$



$$\text{Total: } (\lg n + 1) \cdot n = n \lg n + n = \Theta(n \lg n)$$

(n hojas en el último nivel, cada una con costo $c = 1$)

Figura 1.3: Árbol de recurrencia para Merge Sort. En cada nivel el costo total es n . Como hay $\lg n + 1$ niveles, el costo total es $\Theta(n \lg n)$.

¿Por qué la altura del árbol es $\lg n$?

En cada nivel de recursión, el tamaño del problema se divide a la mitad:

Nivel	Tamaño del subproblema
0	n
1	$n/2$
2	$n/4 = n/2^2$
3	$n/8 = n/2^3$
$\vdots$	$\vdots$
k	$n/2^k$

La recursión termina cuando el tamaño del subproblema llega a 1 (caso base). Buscamos el nivel k donde:

$$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n = \lg n$$

Por lo tanto, el árbol tiene $\lg n$ niveles desde el nivel 1 hasta las hojas. Contando el nivel 0 (la raíz), hay $\lg n + 1$ niveles en total.

Ejemplo 1.1. Si $n = 8$:

- Nivel 0: tamaño 8
- Nivel 1: tamaño 4
- Nivel 2: tamaño 2
- Nivel 3: tamaño 1 (caso base)

Altura = $\lg 8 = 3$ niveles, más la raíz = 4 niveles totales.

Teorema 1.2. Merge Sort tiene complejidad $\Theta(n \lg n)$ en todos los casos.

Demostración. El árbol de recurrencia tiene $\lg n + 1$ niveles (ya que dividimos a la mitad en cada nivel hasta llegar a subproblemas de tamaño 1). En cada nivel, el trabajo total es n (la suma de los costos de todos los nodos en ese nivel). Por lo tanto:

$$T(n) = n \cdot (\lg n + 1) = n \lg n + n = \Theta(n \lg n)$$

□

1.4 Bubble Sort

Bubble Sort es uno de los algoritmos de ordenamiento más simples. Funciona intercambiando repetidamente elementos adyacentes que están en el orden incorrecto. El nombre proviene de la forma en que los elementos más grandes “burbujean” hacia el final del arreglo.

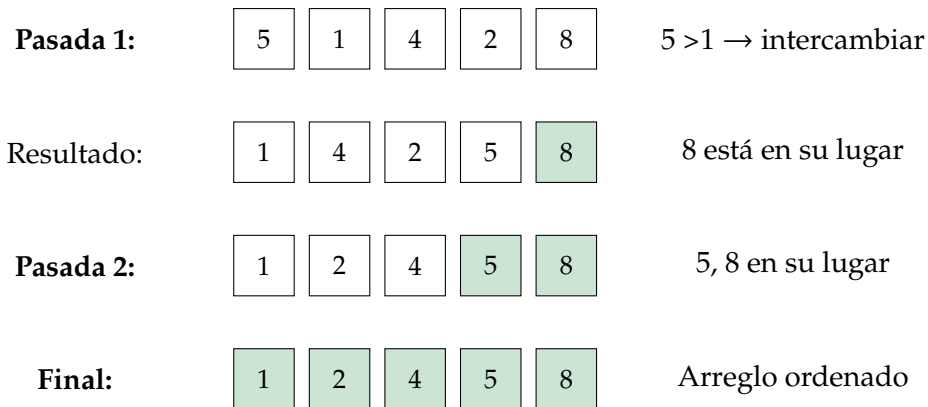


Figura 1.4: Bubble Sort: en cada pasada, el elemento más grande “burbujea” a su posición final.

1.4.1 Pseudocódigo

Algorithm 4 Bubble-Sort(A, n)

```

1: for  $i = 1$  to  $n - 1$  do
2:   for  $j = 1$  to  $n - i$  do
3:     if  $A[j] > A[j + 1]$  then
4:       intercambiar  $A[j]$  con  $A[j + 1]$ 
5:     end if
6:   end for
7: end for

```

1.4.2 Optimización con bandera

Podemos optimizar Bubble Sort detectando si en una pasada no hubo intercambios, lo que indica que el arreglo ya está ordenado:

Algorithm 5 Bubble-Sort-Optimizado(A, n)

```

1: for  $i = 1$  to  $n - 1$  do
2:    $swapped = \text{false}$ 
3:   for  $j = 1$  to  $n - i$  do
4:     if  $A[j] > A[j + 1]$  then
5:       intercambiar  $A[j]$  con  $A[j + 1]$ 
6:        $swapped = \text{true}$ 
7:     end if
8:   end for
9:   if  $not\ swapped$  then
10:    break {Ya está ordenado}
11:   end if
12: end for

```

1.4.3 Análisis de complejidad

- **Peor caso:** $O(n^2)$ - arreglo en orden inverso
- **Mejor caso:** $O(n)$ - arreglo ya ordenado (con optimización)
- **Caso promedio:** $O(n^2)$

1.5 Selection Sort

Selection Sort funciona seleccionando repetidamente el elemento mínimo de la parte no ordenada y moviéndolo al principio.

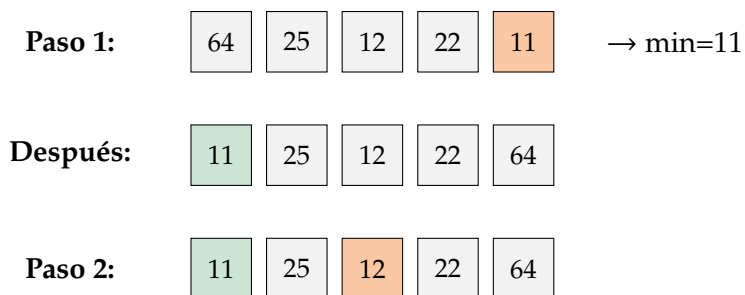


Figura 1.5: Selection Sort: en cada paso, se encuentra el mínimo de la parte no ordenada.

1.5.1 Pseudocódigo

Algorithm 6 Selection-Sort(A, n)

```

1: for  $i = 1$  to  $n - 1$  do
2:    $min\_idx = i$ 
3:   for  $j = i + 1$  to  $n$  do
4:     if  $A[j] < A[min\_idx]$  then
5:        $min\_idx = j$ 
6:     end if
7:   end for
8:   intercambiar  $A[i]$  con  $A[min\_idx]$ 
9: end for

```

1.5.2 Implementación en Python

```

1 def selection_sort(arr):
2     n = len(arr)
3     for i in range(n - 1):
4         min_idx = i
5         for j in range(i + 1, n):
6             if arr[j] < arr[min_idx]:
7                 min_idx = j
8         arr[i], arr[min_idx] = arr[min_idx], arr[i]
9     return arr

```

1.5.3 Análisis de complejidad

- **Comparaciones:** Siempre $\frac{n(n-1)}{2} = O(n^2)$
- **Intercambios:** Exactamente $n - 1 = O(n)$
- **Mejor, peor y promedio:** $O(n^2)$

Ventaja: Minimiza el número de intercambios (útil cuando los intercambios son costosos).

Desventaja: No es adaptativo (siempre hace $O(n^2)$ comparaciones).

1.6 Quick Sort

Quick Sort es un algoritmo de ordenamiento que, al igual que Merge Sort, aplica el paradigma de divide y vencerás. Sin embargo, a diferencia de Merge Sort, todo el trabajo real se hace en la fase de división:

1. **Dividir:** Particionar el arreglo $A[p..r]$ en dos subarreglos $A[p..q - 1]$ y $A[q + 1..r]$ tales que cada elemento de $A[p..q - 1]$ es menor o igual que $A[q]$, que a su vez es menor o igual que cada elemento de $A[q + 1..r]$.
2. **Conquistar:** Ordenar los dos subarreglos recursivamente.
3. **Combinar:** No se requiere trabajo; el arreglo ya está ordenado.

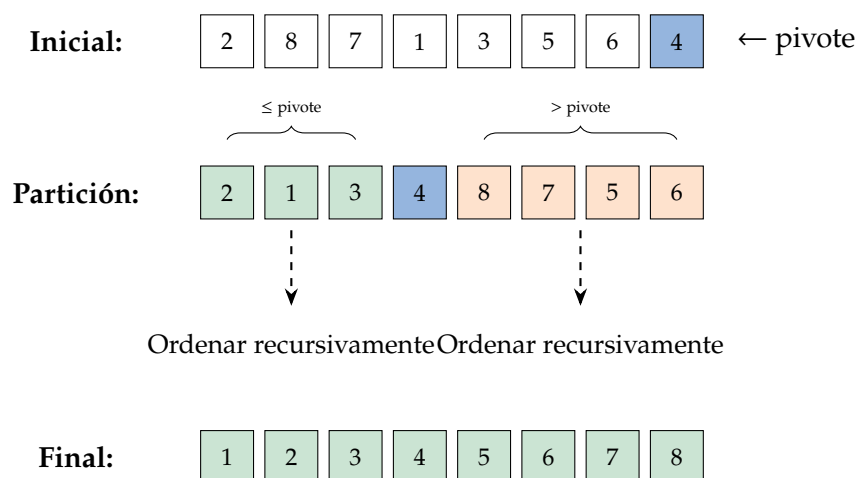


Figura 1.6: Quick Sort: el pivote (4) divide el arreglo en elementos menores y mayores.

1.6.1 Pseudocódigo

Algorithm 7 Quick-Sort(A, p, r)

```

1: if  $p < r$  then
2:    $q = \text{Partition}(A, p, r)$ 
3:   Quick-Sort( $A, p, q - 1$ )
4:   Quick-Sort( $A, q + 1, r$ )
5: end if

```

La clave de Quick Sort es el procedimiento PARTITION:

Algorithm 8 Partition(A, p, r)

```

1:  $x = A[r]$  {El pivote}
2:  $i = p - 1$ 
3: for  $j = p$  to  $r - 1$  do
4:   if  $A[j] \leq x$  then
5:      $i = i + 1$ 
6:     intercambiar  $A[i]$  con  $A[j]$ 
7:   end if
8: end for
9: intercambiar  $A[i + 1]$  con  $A[r]$ 
10: return  $i + 1$ 

```

1.6.2 Análisis de complejidad

El rendimiento de Quick Sort depende de qué tan balanceadas sean las particiones:

- **Peor caso:** $\Theta(n^2)$ - cuando las particiones son maximalmente desbalanceadas (ej. arreglo ya ordenado con pivote al final).
- **Mejor caso:** $\Theta(n \lg n)$ - cuando las particiones son perfectamente balanceadas.

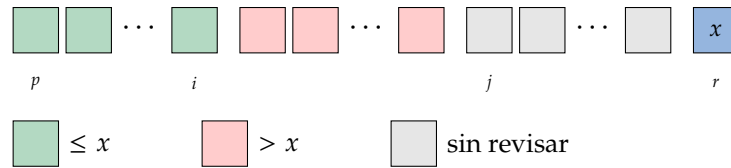
Invariante de Partition:

Figura 1.7: Invariante del procedimiento Partition: elementos a la izquierda de i son $\leq x$, elementos entre i y j son $> x$.

- **Caso promedio:** $\Theta(n \lg n)$ - asumiendo que todas las permutaciones de entrada son igualmente probables.

Teorema 1.3. El tiempo de ejecución esperado de Quick Sort es $O(n \lg n)$.

1.7 Heap Sort

Heap Sort introduce una estructura de datos llamada **heap** (montículo), que es útil tanto para ordenamiento como para implementar colas de prioridad.

1.7.1 La estructura de datos Heap

Un **heap binario** es un arreglo que podemos ver como un árbol binario casi completo. Cada nodo del árbol corresponde a un elemento del arreglo.

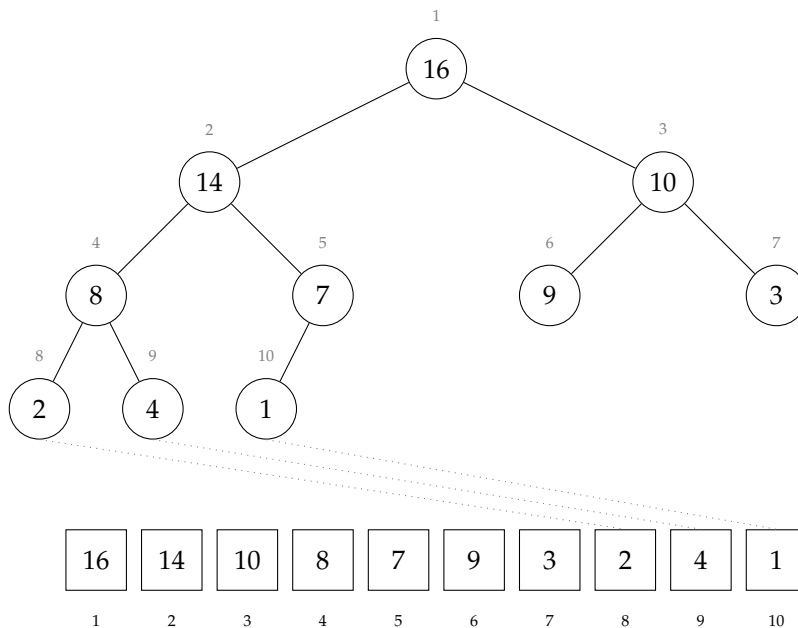


Figura 1.8: Un max-heap visto como árbol binario y como arreglo. El valor de cada nodo es $\geq$ que los valores de sus hijos.

Para un nodo en el índice i :

- **Padre:** $\lfloor i/2 \rfloor$

- **Hijo izquierdo:** $2i$
- **Hijo derecho:** $2i + 1$

Definición 1.1 (Propiedad de Max-Heap). *Para todo nodo i diferente de la raíz: $A[\text{Parent}(i)] \geq A[i]$. Es decir, el valor de un nodo es a lo más el valor de su padre. El elemento más grande está en la raíz.*

1.7.2 Operación Max-Heapify

MAX-HEAPIFY es el procedimiento clave para mantener la propiedad de max-heap. Asume que los subárboles izquierdo y derecho de un nodo i son max-heaps, pero $A[i]$ podría ser menor que sus hijos.

Algorithm 9 Max-Heapify(A, i)

```

1:  $l = \text{Left}(i)$ 
2:  $r = \text{Right}(i)$ 
3: if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$  then
4:    $largest = l$ 
5: else
6:    $largest = i$ 
7: end if
8: if  $r \leq A.\text{heap-size}$  and  $A[r] > A[largest]$  then
9:    $largest = r$ 
10: end if
11: if  $largest \neq i$  then
12:   intercambiar  $A[i]$  con  $A[largest]$ 
13:   Max-Heapify( $A, largest$ )
14: end if

```

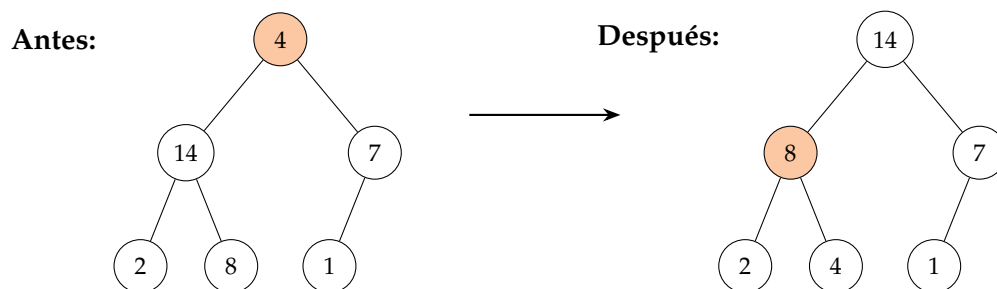


Figura 1.9: Max-Heapify: el valor 4 “baja” intercambiándose con el mayor de sus hijos hasta restaurar la propiedad de heap.

1.7.3 Algoritmo Heap Sort

Algorithm 10 Heap-Sort(A, n)

```

1: Build-Max-Heap( $A, n$ ) {Construir el heap}
2: for  $i = n$  downto 2 do
3:   intercambiar  $A[1]$  con  $A[i]$  {Mover máximo al final}
4:    $A.heap-size = A.heap-size - 1$ 
5:   Max-Heapify( $A, 1$ ) {Restaurar propiedad}
6: end for

```

Teorema 1.4. *Heap Sort tiene complejidad $O(n \lg n)$ en todos los casos.*

1.8 Límite inferior para ordenamiento por comparaciones

¿Podemos hacer algo mejor que $O(n \lg n)$? Para algoritmos basados en comparaciones, la respuesta es no.

Teorema 1.5. *Cualquier algoritmo de ordenamiento basado en comparaciones requiere $\Omega(n \lg n)$ comparaciones en el peor caso.*

Demostración. Un algoritmo basado en comparaciones puede verse como un árbol de decisión binario. Con $n!$ permutaciones posibles, el árbol debe tener al menos $n!$ hojas. Un árbol binario con L hojas tiene altura al menos $\lg L$. Por lo tanto:

$$h \geq \lg(n!) = \Omega(n \lg n)$$

usando la aproximación de Stirling: $n! > (n/e)^n$. □

1.9 Ordenamiento en tiempo lineal

Los algoritmos que no se basan en comparaciones pueden superar el límite de $\Omega(n \lg n)$.

1.9.1 Counting Sort

Counting Sort asume que los n elementos de entrada son enteros en el rango $[0, k]$. Cuando $k = O(n)$, el ordenamiento corre en tiempo $\Theta(n)$.

Algorithm 11 Counting-Sort(A, B, n, k)

```

1: Crear arreglo  $C[0..k]$ 
2: for  $i = 0$  to  $k$  do
3:    $C[i] = 0$ 
4: end for
5: for  $j = 1$  to  $n$  do
6:    $C[A[j]] = C[A[j]] + 1$  { $C[i] = \#$  elementos iguales a  $i$ }
7: end for
8: for  $i = 1$  to  $k$  do
9:    $C[i] = C[i] + C[i - 1]$  { $C[i] = \#$  elementos  $\leq i$ }
10: end for
11: for  $j = n$  downto 1 do
12:    $B[C[A[j]]] = A[j]$ 
13:    $C[A[j]] = C[A[j]] - 1$ 
14: end for

```

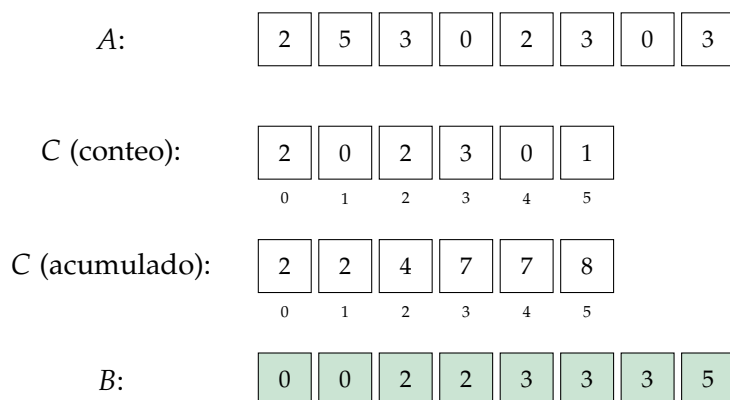


Figura 1.10: Counting Sort: $C[i]$ cuenta cuántos elementos son $\leq i$, lo que determina la posición final.

Counting Sort es **estable**: elementos con el mismo valor aparecen en el arreglo de salida en el mismo orden que en el de entrada.

1.9.2 Radix Sort

Radix Sort ordena números de d dígitos procesando un dígito a la vez, del menos significativo al más significativo.

Algorithm 12 Radix-Sort(A, d)

- 1: **for** $i = 1$ **to** d **do**
 - 2: usar un ordenamiento estable para ordenar A por el dígito i
 - 3: **end for**
-

Original	Por unidades	Por decenas	Por centenas
329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839
	→	→	→

Figura 1.11: Radix Sort ordena números de 3 dígitos procesando del dígito menos significativo al más significativo.

Teorema 1.6. Dado n números de d dígitos donde cada dígito puede tomar hasta k valores posibles, Radix Sort ordena correctamente en tiempo $\Theta(d(n + k))$.

Algoritmo	Mejor	Promedio	Peor	Estable
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sí
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sí
Merge Sort	$O(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$	Sí
Quick Sort	$O(n \lg n)$	$O(n \lg n)$	$O(n^2)$	No
Heap Sort	$O(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	Sí
Radix Sort	$O(dn)$	$O(dn)$	$O(dn)$	Sí

Cuadro 1.1: Comparación de complejidades temporales de algoritmos de ordenamiento.

1.10 Comparación de algoritmos

1.11 Ejercicios

1.11.1 Ejercicios Teóricos

Insertion Sort

1. Ilustra la operación de Insertion Sort sobre el arreglo $A = \langle 31, 41, 59, 26, 41, 58 \rangle$.
2. Reescribe el procedimiento Insertion-Sort para ordenar en orden no creciente en lugar de no decreciente.
3. Considera un arreglo de n elementos donde los primeros $n - 1$ elementos ya están ordenados. ¿Cuál es la complejidad de Insertion Sort en este caso? Justifica tu respuesta.
4. **Búsqueda binaria en Insertion Sort:** Modifica Insertion Sort para usar búsqueda binaria al encontrar la posición de inserción. ¿Mejora esto la complejidad temporal? ¿Por qué sí o por qué no?

Merge Sort

5. Usando la Figura 1.2 como modelo, ilustra la operación de Merge Sort sobre el arreglo $A = \langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$.
6. Escriba la recurrencia para el tiempo de ejecución de Merge Sort cuando el arreglo se divide en tercios en lugar de mitades. ¿Cuál es la solución de esta recurrencia?
7. **Inversiones:** Una *inversión* en un arreglo $A[1..n]$ es un par (i, j) tal que $i < j$ y $A[i] > A[j]$. Modifica Merge Sort para contar el número de inversiones en tiempo $\Theta(n \lg n)$.
8. Demuestra que cualquier algoritmo de ordenamiento basado en comparaciones requiere $\Omega(n \lg n)$ comparaciones en el peor caso.

Bubble Sort

9. Demuestre que la versión optimizada de Bubble Sort tiene tiempo de ejecución $\Omega(n)$ en el mejor caso.
10. ¿Cuántos intercambios realiza Bubble Sort en el peor caso para un arreglo de n elementos? Demuéstralo.
11. **Cocktail Shaker Sort:** Investiga e implementa una variante de Bubble Sort llamada “Cocktail Shaker Sort” que alterna la dirección de las pasadas. ¿Mejora el rendimiento en algún caso?

Quick Sort

12. Ilustre la operación de Partition sobre el arreglo $A = \langle 13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11 \rangle$.
13. ¿Cuál es el tiempo de ejecución de Quick Sort cuando todos los elementos del arreglo tienen el mismo valor?
14. **Mediana de tres:** Implementa una versión de Quick Sort que elija el pivote como la mediana de tres elementos: el primero, el del medio y el último. ¿Cómo afecta esto al rendimiento en arreglos ya ordenados?
15. Demuestra que la profundidad de la pila de recursión de Quick Sort en el peor caso es $\Theta(n)$. ¿Cómo se puede modificar Quick Sort para garantizar que la profundidad sea $O(\lg n)$?
16. **Quick Sort con 3 particiones:** Diseña una versión de Quick Sort que particione el arreglo en tres partes: elementos menores que el pivote, iguales al pivote, y mayores que el pivote. ¿Cuál es la ventaja de esta versión?

Heap Sort

17. Usando la Figura 1.8 como modelo, ilustre la operación de Build-Max-Heap sobre el arreglo $A = \langle 5, 3, 17, 10, 84, 19, 6, 22, 9 \rangle$.
18. Demuestra que hay a lo más $\lceil n/2^{h+1} \rceil$ nodos de altura h en cualquier heap de n elementos.
19. **Min-Heap:** Escribe los procedimientos Min-Heapify, Build-Min-Heap, y Heap-Extract-Min.
20. ¿Es el arreglo ordenado en orden decreciente un max-heap válido? Justifica tu respuesta.
21. **Heap d -ario:** Generaliza el heap binario a un heap d -ario donde cada nodo tiene d hijos. ¿Cuál es la altura de un heap d -ario de n elementos? ¿Cómo afecta d a la complejidad de las operaciones?

Counting Sort y Radix Sort

22. ¿Por qué queremos que el ordenamiento auxiliar usado en Radix Sort sea estable?
23. Suponga que tiene acceso a una subrutina de ordenamiento en tiempo lineal que ordena n elementos en el rango $[1, n^2]$. Describa un algoritmo que ordene n enteros arbitrarios en $O(n)$ usando esta subrutina.
24. **Counting Sort para rangos negativos:** Modifica Counting Sort para manejar enteros en el rango $[-k, k]$.
25. Si tienes n números en el rango $[0, n^3 - 1]$, ¿cuántos dígitos base n tiene cada número? ¿Cuál es la complejidad de Radix Sort en este caso?

1.11.2 Ejercicios de Programación

1. Implementa todos los algoritmos de ordenamiento vistos en este capítulo en tu lenguaje de programación favorito.
2. Escribe un programa que compare experimentalmente el tiempo de ejecución de los diferentes algoritmos para arreglos de diferentes tamaños y distribuciones (aleatorio, ordenado, inversamente ordenado).
3. Implementa una versión iterativa de Merge Sort usando una pila explícita.
4. Implementa Heap Sort usando un min-heap para ordenar en orden ascendente.

1.11.3 Problemas de LeetCode

Los siguientes problemas de LeetCode permiten practicar la aplicación de algoritmos de ordenamiento:

Nivel Fácil

1. **LeetCode 912 - Sort an Array**¹

Dado un arreglo de enteros `nums`, ordénalo en orden ascendente y devuélvelo.

Hint: Implementa Merge Sort o Quick Sort. No uses la función de ordenamiento integrada.

2. **LeetCode 88 - Merge Sorted Array**²

Dados dos arreglos ordenados `nums1` y `nums2`, mezcla `nums2` en `nums1` como un solo arreglo ordenado.

Hint: Usa la técnica de merge de Merge Sort, pero trabajando desde el final hacia el inicio.

3. **LeetCode 217 - Contains Duplicate**³

Dado un arreglo de enteros, determina si contiene duplicados.

Hint: Ordena el arreglo y busca elementos adyacentes iguales.

4. **LeetCode 242 - Valid Anagram**⁴

Dadas dos cadenas `s` y `t`, determina si `t` es un anagrama de `s`.

Hint: Ordena ambas cadenas y compáralas, o usa Counting Sort sobre los caracteres.

Nivel Medio

5. **LeetCode 75 - Sort Colors**⁵

Dado un arreglo con n objetos coloreados de rojo (0), blanco (1) o azul (2), ordénalos in-place.

Hint: Este es el problema de la bandera holandesa. Usa particionamiento similar a Quick Sort con 3 regiones.

6. **LeetCode 148 - Sort List**⁶

Dada la cabeza de una lista enlazada, devuélvela ordenada en orden ascendente.

Hint: Implementa Merge Sort para listas enlazadas. Usa slow / fast pointers para encontrar el punto medio.

7. **LeetCode 179 - Largest Number**⁷

Dado una lista de enteros no negativos, arregla los números para formar el número más grande.

Hint: Define un comparador personalizado: compara $a + b$ vs $b + a$ como strings.

¹<https://leetcode.com/problems/sort-an-array/>

²<https://leetcode.com/problems/merge-sorted-array/>

³<https://leetcode.com/problems/contains-duplicate/>

⁴<https://leetcode.com/problems/valid-anagram/>

⁵<https://leetcode.com/problems/sort-colors/>

⁶<https://leetcode.com/problems/sort-list/>

⁷<https://leetcode.com/problems/largest-number/>

8. **LeetCode 215 - Kth Largest Element in an Array**⁸

Encuentra el k -ésimo elemento más grande en un arreglo desordenado.

Hint: Usa un min-heap de tamaño k , o el algoritmo Quick Select (variante de Quick Sort).

9. **LeetCode 347 - Top K Frequent Elements**⁹

Dado un arreglo de enteros y un entero k , devuelve los k elementos más frecuentes.

Hint: Usa un heap para mantener los k elementos más frecuentes, o Bucket Sort por frecuencia.

10. **LeetCode 56 - Merge Intervals**¹⁰

Dado un arreglo de intervalos, mezcla todos los intervalos superpuestos.

Hint: Ordena los intervalos por tiempo de inicio, luego itera mezclando intervalos superpuestos.

11. **LeetCode 973 - K Closest Points to Origin**¹¹

Dado un arreglo de puntos, encuentra los k puntos más cercanos al origen.

Hint: Usa un max-heap de tamaño k , o Quick Select.

Nivel Difícil

12. **LeetCode 315 - Count of Smaller Numbers After Self**¹²

Dado un arreglo de enteros, devuelve un nuevo arreglo `counts` donde `counts[i]` es el número de elementos más pequeños a la derecha de `nums[i]`.

Hint: Modifica Merge Sort para contar inversiones mientras ordenas.

13. **LeetCode 493 - Reverse Pairs**¹³

Dado un arreglo `nums`, devuelve el número de pares inversos importantes. Un par inverso importante es un par (i, j) donde $i < j$ y $nums[i] > 2 \cdot nums[j]$.

Hint: Similar al conteo de inversiones con Merge Sort, pero con condición modificada.

14. **LeetCode 327 - Count of Range Sum**¹⁴

Dado un arreglo de enteros y un rango $[lower, upper]$, devuelve el número de sumas de rango que caen dentro del rango.

Hint: Usa sumas prefijo y Merge Sort para contar pares que cumplen la condición.

15. **LeetCode 23 - Merge k Sorted Lists**¹⁵

Mezcla k listas enlazadas ordenadas en una sola lista enlazada ordenada.

Hint: Usa un min-heap para siempre obtener el siguiente elemento más pequeño, o aplica la técnica de merge divide-and-conquer.

⁸<https://leetcode.com/problems/kth-largest-element-in-an-array/>

⁹<https://leetcode.com/problems/top-k-frequent-elements/>

¹⁰<https://leetcode.com/problems/merge-intervals/>

¹¹<https://leetcode.com/problems/k-closest-points-to-origin/>

¹²<https://leetcode.com/problems/count-of-smaller-numbers-after-self/>

¹³<https://leetcode.com/problems/reverse-pairs/>

¹⁴<https://leetcode.com/problems/count-of-range-sum/>

¹⁵<https://leetcode.com/problems/merge-k-sorted-lists/>

1.11.4 Problemas Adicionales de Análisis

1. **Límite inferior de ordenamiento:** Demuestra formalmente que cualquier algoritmo de ordenamiento basado en comparaciones requiere al menos $\lceil \lg(n!) \rceil$ comparaciones en el peor caso. Usa la aproximación de Stirling para mostrar que esto es $\Omega(n \lg n)$.
2. **Estabilidad:** Para cada algoritmo de este capítulo, determina si es estable o no. Si no es estable, proporciona un contraejemplo.
3. **Ordenamiento adaptativo:** Un algoritmo es *adaptativo* si su rendimiento mejora cuando la entrada ya está parcialmente ordenada. ¿Cuáles de los algoritmos estudiados son adaptativos?
4. **Trade-offs de espacio:** Compara el uso de memoria adicional de cada algoritmo. ¿Cuál usarías si la memoria es muy limitada?

1.12 Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4ta ed.). MIT Press. Capítulos 2, 6, 7, 8.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4ta ed.). Addison-Wesley. Capítulo 2.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2da ed.). Addison-Wesley.

1.13 Apéndice: Código de Soporte

Esta sección documenta las funciones de utilidad disponibles en el directorio `src/` del repositorio. Estas funciones están diseñadas para facilitar la experimentación y prueba de los algoritmos de ordenamiento.

1.13.1 Estructura del directorio

```
01-ordenamiento/
+-- src/
    +-- __init__.py
    +-- main.py          # Implementaciones de algoritmos
    +-- test_utils.py    # Utilidades para pruebas
    +-- utils_general.py # Utilidades generales
```

1.13.2 Módulo `test_utils.py`

Este módulo proporciona funciones para generar datos de prueba y medir el rendimiento de los algoritmos.

`generate_random_data(size: int) -> List[int]`

Genera una lista de números enteros aleatorios en el rango $[0, 10000]$.

```
1 from test_utils import generate_random_data
2
3 # Generar 1000 numeros aleatorios
4 datos = generate_random_data(1000)
5 print(f"Primeros 10: {datos[:10]}")
```

- **Parámetro:** `size` – Tamaño de la lista a generar.

- **Retorna:** Lista de enteros aleatorios.
- **Uso típico:** Probar algoritmos con entrada aleatoria (caso promedio).

generate_nearly_sorted(size: int, swaps: int = None) ->List[int]

Genera una lista casi ordenada realizando algunos intercambios aleatorios sobre una lista ordenada.

```
1 from test_utils import generate_nearly_sorted
2
3 # Lista casi ordenada con pocos intercambios
4 datos = generate_nearly_sorted(1000, swaps=10)
```

- **Parámetros:**
 - size – Tamaño de la lista.
 - swaps – Número de intercambios (default: size // 10).
- **Retorna:** Lista casi ordenada.
- **Uso típico:** Evaluar algoritmos adaptativos como Insertion Sort.

generate_reversed(size: int) ->List[int]

Genera una lista ordenada en orden inverso (descendente).

```
1 from test_utils import generate_reversed
2
3 # Lista [1000, 999, 998, ..., 2, 1]
4 datos = generate_reversed(1000)
```

- **Parámetro:** size – Tamaño de la lista.
- **Retorna:** Lista ordenada inversamente.
- **Uso típico:** Probar el peor caso de algoritmos como Quick Sort (sin aleatorización).

measure_time(func: Callable, arr: List[int]) ->float

Mide el tiempo de ejecución de un algoritmo de ordenamiento.

```
1 from test_utils import measure_time, generate_random_data
2 from main import merge_sort, quick_sort
3
4 datos = generate_random_data(10000)
5
6 # Medir tiempo de Merge Sort
7 tiempo_merge = measure_time(merge_sort, datos)
8 print(f"Merge Sort: {tiempo_merge:.4f} segundos")
9
10 # Medir tiempo de Quick Sort
11 tiempo_quick = measure_time(quick_sort, datos)
12 print(f"Quick Sort: {tiempo_quick:.4f} segundos")
```

- **Parámetros:**

- `func` – Función de ordenamiento a medir.
- `arr` – Lista a ordenar.

- **Retorna:** Tiempo de ejecución en segundos.

- **Nota:** La función crea una copia del arreglo para no modificar el original.

1.13.3 Módulo `main.py`

Contiene los esqueletos de los algoritmos de ordenamiento para que los implementes.

Función	Descripción
<code>bubble_sort(arr)</code>	Bubble Sort básico
<code>merge_sort(arr)</code>	Merge Sort recursivo
<code>quick_sort(arr)</code>	Quick Sort con pivote
<code>heapify(arr, n, i)</code>	Auxiliar para Heap Sort
<code>heap_sort(arr)</code>	Heap Sort usando max-heap
<code>counting_sort(arr)</code>	Counting Sort para enteros
<code>counting_sort_for_radix(arr, exp)</code>	Counting Sort modificado
<code>radix_sort(arr)</code>	Radix Sort LSD

Cuadro 1.2: Funciones disponibles en `main.py`

1.13.4 Ejemplo: Comparación experimental

El siguiente script compara todos los algoritmos:

```

1 from test_utils import (generate_random_data, generate_nearly_sorted,
2                           generate_reversed, measure_time)
3 from main import (bubble_sort, merge_sort, quick_sort,
4                   heap_sort, counting_sort, radix_sort)
5
6 # Configuración
7 SIZES = [100, 500, 1000, 5000]
8 algorithms = {
9     'Bubble Sort': bubble_sort,
10    'Merge Sort': merge_sort,
11    'Quick Sort': quick_sort,
12    'Heap Sort': heap_sort,
13 }
14
15 # Experimento
16 for size in SIZES:
17     print(f"\n=== Tamaño: {size} ===")
18     datos = generate_random_data(size)
19
20     for name, func in algorithms.items():
21         tiempo = measure_time(func, datos)
22         print(f"{name}: {tiempo:.4f}s")

```

1.13.5 Verificación de correctitud

Para verificar que tu implementación es correcta:

```
1 from test_utils import generate_random_data
2 from main import merge_sort # o cualquier algoritmo
3
4 def verificar_ordenamiento(func, n=100, pruebas=10):
5     """Verifica que un algoritmo ordena correctamente."""
6     for _ in range(pruebas):
7         datos = generate_random_data(n)
8         esperado = sorted(datos)
9         resultado = func(datos.copy())
10        assert resultado == esperado, "Error en ordenamiento!"
11        print(f"{func.__name__} paso todas las pruebas")
12
13 verificar_ordenamiento(merge_sort)
```